

SQL Set Operators

3 · Set Operators — Complete Reference Guide (Banking Project)

Table of Contents

1. UNION
2. UNION ALL
3. EXCEPT
4. INTERSECT

1. UNION

→ **Task 1: Combine Premium-segment and Corporate-segment customers into a single high-value outreach list. UNION is the right choice here because it automatically removes any duplicate rows if a customer were ever double-counted.**

```
SELECT customer_id, first_name, last_name, customer_segment
FROM Customers
WHERE customer_segment = 'Premium'
UNION
SELECT customer_id, first_name, last_name, customer_segment
FROM Customers
WHERE customer_segment = 'Corporate';
```

→ **Task 2: Build a single distinct list of account IDs that received money in -- either through a Deposit or an Interest Credit -- for a 'money-in' event tracker. UNION removes the duplicate account_id rows for accounts that received both types.**

```
SELECT account_id
FROM Transactions
WHERE transaction_type = 'Deposit'
UNION
SELECT account_id
FROM Transactions
WHERE transaction_type = 'Interest Credit';
```

2. UNION ALL

→ **Task 3: Build a combined cash-movement report of every Deposit and every Withdrawal transaction. UNION ALL is the right choice over UNION here -- every transaction_id is already unique, so there is nothing to de-duplicate, and UNION ALL avoids the unnecessary performance cost of checking for duplicates.**

```
SELECT
    transaction_id, account_id, transaction_date,
    transaction_amount, 'Deposit' AS movement_type
FROM Transactions
WHERE transaction_type = 'Deposit'
UNION ALL
SELECT
    transaction_id, account_id, transaction_date,
    transaction_amount, 'Withdrawal' AS movement_type
FROM Transactions
WHERE transaction_type = 'Withdrawal';
```

→ **Task 4: Combine each account's total Deposit amount and total Interest Credit amount into one side-by-side category report. UNION ALL is correct here because two accounts could legitimately have the exact same total -- UNION would incorrectly collapse those matching rows into one.**

```
SELECT account_id, SUM(transaction_amount) AS total_amount,
       'Deposit' AS category
FROM Transactions
WHERE transaction_type = 'Deposit'
GROUP BY account_id
UNION ALL
SELECT account_id, SUM(transaction_amount) AS total_amount,
       'Interest Credit' AS category
FROM Transactions
WHERE transaction_type = 'Interest Credit'
GROUP BY account_id;
```

3. EXCEPT

→ **Task 5: Find customers who own at least one account but have never made a single transaction, for the customer-engagement team's follow-up list. EXCEPT removes every customer that appears in the second (transacting) result set from the first (account-holding) result set.**

```
SELECT c.customer_id, c.first_name, c.last_name
FROM Customers c
INNER JOIN Accounts a
      ON c.customer_id = a.customer_id
EXCEPT
SELECT c.customer_id, c.first_name, c.last_name
FROM Customers c
INNER JOIN Accounts a
      ON c.customer_id = a.customer_id
INNER JOIN Transactions t
      ON a.account_id = t.account_id;
```

→ **Task 6: Find every account that has never had a single Successful transaction, for the account-health review. EXCEPT subtracts the set of accounts with at least one Success transaction from the complete set of all accounts.**

```
SELECT account_id, account_type
FROM Accounts
EXCEPT
SELECT a.account_id, a.account_type
FROM Accounts a
INNER JOIN Transactions t
      ON a.account_id = t.account_id
WHERE t.transaction_status = 'Success';
```

4. INTERSECT

→ **Task 7: Find customers who are BOTH in the Premium segment AND have made at least one transaction through the Mobile App channel, for a digital-engagement campaign targeted only at Premium customers already using the app.**

```
SELECT c.customer_id, c.first_name, c.last_name
FROM Customers c
WHERE c.customer_segment = 'Premium'
INTERSECT
SELECT c.customer_id, c.first_name, c.last_name
FROM Customers c
INNER JOIN Accounts a
    ON c.customer_id = a.customer_id
INNER JOIN Transactions t
    ON a.account_id = t.account_id
WHERE t.transaction_channel = 'Mobile App';
```

→ **Task 8: Find accounts that have had BOTH a Deposit AND a Withdrawal, to identify 'active bidirectional' accounts that are genuinely being used for day-to-day banking rather than one-way fund parking.**

```
SELECT account_id
FROM Transactions
WHERE transaction_type = 'Deposit'
INTERSECT
SELECT account_id
FROM Transactions
WHERE transaction_type = 'Withdrawal';
```